

NAME:TANKISO MASOEBE

ADVANCED DATABASES WK 6

1. Scenario Analysis: E-Commerce Database

An e-commerce company manages a large database containing customer information, products, orders, and payments. As the number of customers increases, the database may become slow and difficult to manage. To improve performance and availability, the company can use both replication and partitioning. Replication involves creating copies of the database on different servers. For example, the main database can handle customer orders while replica databases handle product searches and customer requests. If the main server fails, a replica can help maintain service availability. Partitioning divides large tables into smaller sections based on specific criteria. For example, the orders table can be partitioned by year or month, allowing the database to retrieve recent orders faster. This approach also makes data management easier because older partitions can be archived separately. Combining replication and partitioning improves database performance, scalability, availability, and reliability while allowing the e-commerce company to handle a growing number of customers and transactions efficiently.

2. CONCEPT RESEARCH:

CAP Theorem

The CAP theorem explains that a distributed database system cannot guarantee all three properties simultaneously: **Consistency, Availability, and Partition Tolerance**. Consistency means every user receives the most recent and correct data. Availability means the system continues responding to requests even when some components fail. Partition tolerance means the system continues operating despite communication failures between servers. In practice, network partitions can occur, so distributed systems must balance consistency and availability. For example, an online banking system may prioritize consistency to prevent incorrect account balances, while a social media platform may prioritize availability so users can continue accessing content. The CAP theorem helps database designers choose an appropriate architecture based on application requirements, reliability, and performance.

3.TOOL PRACTICE:

In PostgreSQL, I practiced partitioning a large table to improve database performance and management. I created an **orders** table and partitioned it by the order date using **RANGE partitioning**. Separate partitions were created for different years, such as 2024, 2025, and 2026. When a new order is inserted, PostgreSQL automatically places it into the correct partition based on its date. This practice showed that partitioning

makes it easier to manage large amounts of data because queries can access only the relevant partition instead of the entire table. It can also improve query performance and simplify data archiving. However, partitioning requires proper planning because incorrect partition boundaries can cause errors or inefficient data distribution. Overall, the practical exercise demonstrated how PostgreSQL partitioning can improve scalability, organization, and performance when working with large databases.

4.DIAGRAM:

